GLASS BOX

An options agent whose every decision you can verify

Alpaca AI Trading Agents Hackathon 2026 | Options Alpha Agents | Nilay Toshniwal, solo

1,390
sealed artifacts, one Merkle tree

193 of 197
opportunities declined, with reasons

233
automated tests

1
bug the log caught in its author

`make verify` recomputes the root over every decision. You do not have to trust the operator.

root e4ce452c697f8c90...

Most trading agents are built to trade

This one is built to measure first, and to decline out loud when the premium is not there.

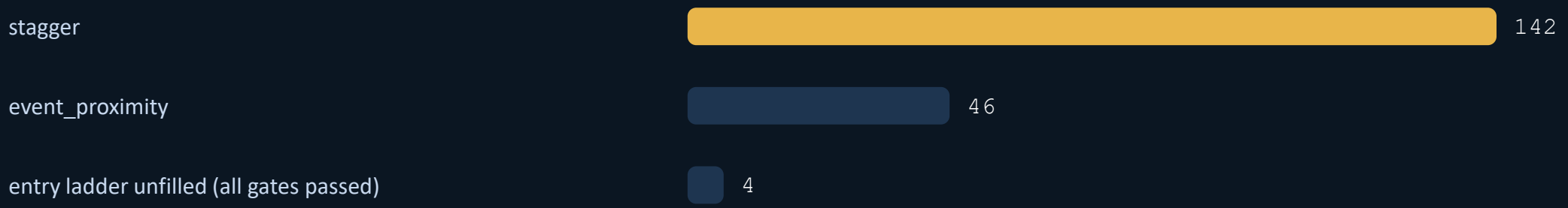
98%

of real opportunities declined

A refusal is logged with the same detail as a fill: every gate, every measured number, sealed into the same tree.

That is the product. An agent that cannot say no is not risk-managed, it is just fast.

What blocked the trades, counted from the log



Registered before the code existed

Provable from git history, not asserted on a slide. The hypotheses, the trial count and the significance bar were all committed before a single line of backtest ran.

+3.68

mean volatility risk premium, vol points

t = +4.74

Newey-West corrected, 1,741 observations

6.99 yrs

SPY, out of sample

The naive number is 18.16, and it is wrong

21-day forward windows sampled daily overlap by 20 of 21 days, so the observations are not independent. Reporting 18.16 would have been four times more impressive and meaningless.

Seven trials, one corrected bar

The significance bar was recomputed to 0.791 as trials were added, and set before any result was seen. The deployed tenor had to clear the bar that the search itself raised.

What did not work

Reported with the same weight as the wins, because a submission that only reports its wins is a sales document. All four are in the repo.

A registered prediction was wrong

P2 predicted the regime gate would show a 1.0 point mean advantage. Measured 0.59. The mean was the wrong statistic; the gate survives because the premium is reliably present in contango, not because contango pays more.

The pricing model failed its own gate

26.78% median error against a pre-registered 15% threshold, so H3 was not run while it stood. VIX is a variance-swap rate, not ATM implied vol. Corrected on 2024 data only, then passed at 10.75%.

The deployed result is fragile, and we say so

Doubling transaction costs improved the best trial's Sharpe, which is impossible and means the optimiser is partly selecting noise. The ungated baseline beats the gated variants on average. Disclosed, not buried.

Our own cost model was lying

Charging cost as a percentage of credit under-charged exactly the configurations that looked best. Measured: the spread is 0.8% of a 35-delta option's price and 13.3% of a 5-delta option's. A 16x range.

Eleven gates, then a trade

Evaluated at decision time, all of them, so one artifact shows the whole picture rather than the first failure.

0 position integrity	1 session window	**2 drawdown breaker**	**3 daily loss limit**
4 consecutive losses	5 capacity	6 term structure	7 VRP threshold
8 event proximity	9 cost ceiling	10 sizing	+ stagger rule

Gates 0, 2 and 3 are circuit breakers, outlined above and distinguished in code: a refusal means no trade now, a breach means stop.

Sizing is a recorded deviation, not a default

Research ran at 1% per position. Live runs 5 concurrent at 3%, hard capped at 15%, because research sizing implied under one trade all week. Written down before the week started.

The deadline is a risk event

A dedicated mechanism flattens the entire book 90 minutes before submission, so the reported result is realised P&L and not a mark at an arbitrary instant.

Three strategies, one market

Three candidate tenors ran live in parallel for eight days, on the same real prices, risking nothing. The deployed one was picked from that, not from the backtest ranking.

T4

7 to 14 DTE

16

would-enter cycles of 52

T6 DEPLOYED

21 to 45 DTE

26

would-enter cycles of 52

T7

5 to 10 DTE

13

would-enter cycles of 51

The backtest preferred one tenor. Eight days of live paper comparison preferred another, and the live evidence won. **The cost of switching is written down in the same file as the decision.**

Every shadow cycle is a real decision on real prices. None of them ever sends an order.

Built on Alpaca, and provably so

Every runtime decision goes through the MCP server. The CLI reads the account back from a different surface.

MCP server, the execution path

alpaca-mcp-server 2.3.0, JSON-RPC over stdio, 74 tools, all versions pinned after fastmcp 4.0.0 shipped mid-competition and broke the unpinned server at import.

Every request and response is recorded inside the sealed artifact, failures included, and the dashboard shows the calls and latencies for the latest cycle.

CLI, the read-only second opinion

Alpaca's official CLI 0.0.14, behind an allowlist of read commands. order submit, position close-all and order cancel-all are refused before a process starts.

Reading the account back through a different Alpaca surface than the one that wrote to it is a real check, not a box tick.

1.5%

round trip cost, laddered from mid

8.3%

cost if it crossed the spread

5

ladder rungs before it gives up

246 ms

measured API round trip

The log caught its own author

First live morning, 09:45 ET. This is the strongest evidence in the submission.

A payload-parsing bug read the account as flat, so the agent opened four condors where it should have opened one.

The proof was a single sealed artifact holding both the raw broker response, with the legs plainly in it, and the reconciliation that ignored them. The bug could not hide from the record it had just written.

Found

in the artifact, not in a dashboard

Fixed

reconciliation now reads both shapes

Tested

a regression test for that exact payload

Kept

positions adopted, incident written up

RISK_REGISTER.md sections 4.7 and 4.8 name the exact artifact. Nothing was quietly deleted, because the seal would have shown it.

The model explains. The numbers decide.

This is an AI agents hackathon, so the honest question is where the model sits. Ours sits after the decision.

What it does

After a decision is sealed, Qwen 2.5 72B on Featherless reads that artifact and writes three plain sentences a judge can understand.

It runs outside the trading loop, on a schedule. Nothing it produces flows back into the agent. The decision existed, and was sealed, before the text did.

Why you can believe it

Every number in the generated text is checked against the artifact it came from, digit for digit. An explanation that invents or alters a number is rejected and counted, not quietly shown.

A hallucinated sentence under a sealed decision would be worse than no sentence at all.

238

decisions explained

0

rejected by the grounding check

0

model decisions to trade

Try to tamper with it

The verification is not a claim on a slide. It is a button on the dashboard and a command in the repo.

```
sealed root       e4ce452c697f8c90...

honest recompute  e4ce452c697f8c90...   MATCHES

after moving one decision's spot by one cent

                  253b8da69a86901f...   DETECTED
```

SHA-256 Merkle tree with domain-separated leaves and nodes, so a forger who recomputes the leaf hashes consistently still fails. The root is sealed before outcomes are known.

One command, no credentials

make judge runs the tests, recomputes the root, prints the decision summary, and regenerates the results page from the log it just verified.

Or press the button

The dashboard edits one sealed decision in memory and recomputes the root in front of you. Nothing on disk is touched, and the root breaks.

Live week, as it happened

Read from the sealed log when this deck was built: 2026-09-04 07:44 UTC. Rebuild the deck, the numbers move.

$99,435	-$565	4	28
equity, from $100,000	profit and loss	condors open	contracts

Bounded by construction

$3,108 of credit collected against a worst case of $10,840, which is 10.8% of the account. Defined risk means the loss is known before the order is sent.

Unattended and supervised

Every cycle runs in its own process with an OS-enforced timeout, because an in-process timer cannot rescue a thread stuck in a syscall. The watchdog restarts what it kills.

One week is mostly noise

The backtest does not predict this week. It establishes positive expectancy, so these days are a draw from a favourable distribution, not a coin flip. We report the draw.

The tail hedge is designed, coded and never engaged: no VIX expiry inside its 21 to 45 day window was quoted all week. That refusal is logged every cycle rather than hidden.

Sixty seconds, and you can check all of it

Nothing here needs our credentials, our machine, or our word.

`make test`	233 tests over the gates, the ladder, the flatten and the deadline
`make verify`	recompute the Merkle root over every logged decision
`make summary`	opportunities, entries, and every refusal by blocking gate
`make judge`	all of the above, then rebuild the results page from the log

The dashboard renders entirely from committed artifacts, so it works with the market closed, the API down and the MCP server stopped. A demo that needs a live API is a demo that dies during the demo.

You do not have to trust me

That is the entire design. Clone it and check.

Repository
github.com/nilaymastaadmi/alpaca-hackathon

MIT licensed. Pre-registration, every result including the failures, and the full decision log.

Live dashboard
alpaca-hackathon-wcjwdbkqifybupyd6ckzps.streamlit.app

The agent's decisions, the shadow race, the MCP traffic, and the tamper button.